

Regression and Support Vector Machine

COMP3314

Introduction to Machine Learning

Dong Xu

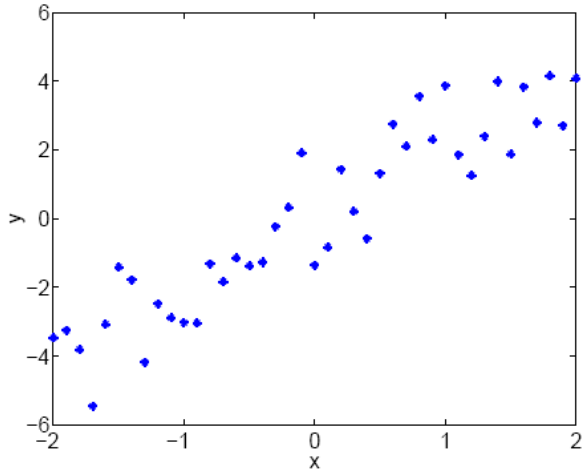
Outline of Lecture

- Linear regression
  - Estimation criterion
  - Least squares solution
  - Generalization and overfitting
  - Extensions
- Support Vector Machine
  - Maximum Margin Hyperplane
  - The Lagrangian Dual Problem
  - The non-separable case & Non-linear SVMs and kernel methods
  - Optimization techniques
- Cross-Validation
- Performance Evaluation

Outline of Lecture

- Linear regression
  - Estimation criterion
  - Least squares solution
  - Generalization and overfitting
  - Extensions
- Support Vector Machine
  - Maximum Margin Hyperplane
  - The Lagrangian Dual Problem
  - The non-separable case & Non-linear SVMs and kernel methods
  - Optimization techniques
- Cross-Validation
- Performance Evaluation

Regression



- The goal is to predict responses/outputs for inputs
- We need to define
  - **1. function class, the type of predictions we consider**
  - **2. fitting criterion (loss) that measures the degree of t to the data**

Linear regression

- Linear functions of one variable (two parameters)

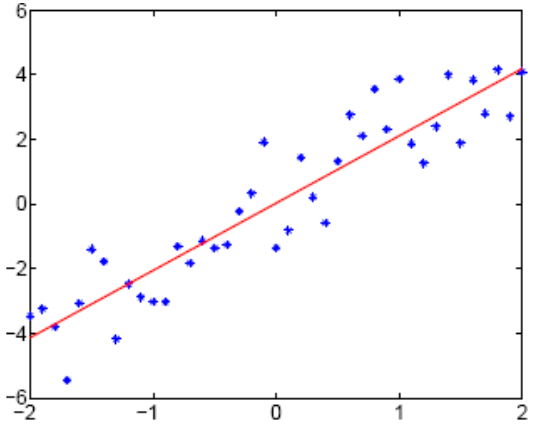
$$f(x\,;\,w) = w_0 + w_1 x;\ \mathrm{w} = [w_0\ w_1]^T$$

and a squared loss: $Loss(\,y,\,f\,(x\,;\,w)) = (y - f(x;\,w))^2/2$.

- Estimation based on minimizing the empirical loss

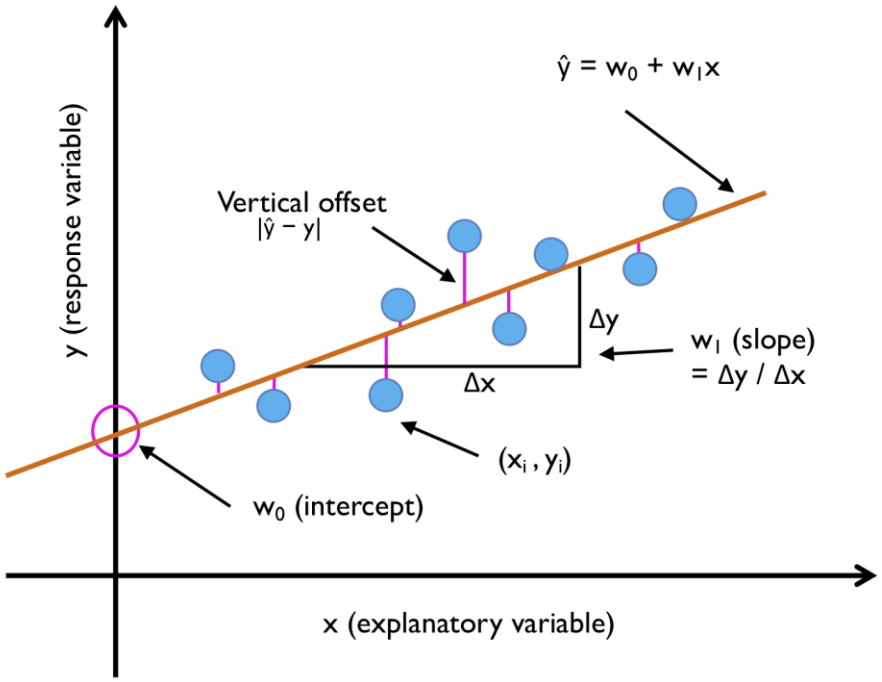
$$J_n(w) = \sum_{i=1}^{n} Loss(y_i, f(x_i; w))$$

with respect to the parameters w.



Linear Regression

- Can be understood as finding the best-fitting straight line through the training examples
- This best-fitting line is also called the regression line, and the vertical lines from the regression line to the training examples are the so-called offsets or residuals—the errors of our prediction



Linear regression: estimation

- We minimize the empirical squared loss

$$J_n(w) = \sum_{i=1}^{n} Loss(y_i; f(x_i; w)) = \sum_{i=1}^{n} (y_i - w_0 - w_1 x_i)^2 / 2$$

Setting the derivatives with respect to w_0 and w_1 to zero we get necessary conditions for the "optimal" parameter values

$$\frac{\partial}{\partial w_0} J_n(w) = -\sum_{i=1}^{n} (y_i - w_0 - w_1 x_i) = 0$$

$$\frac{\partial}{\partial w_1} J_n(w) = -\sum_{i=1}^{n} (y_i - w_0 - w_1 x_i) x_i = 0$$

- Note: These conditions mean that the prediction error (y_i-w_0-$w_1 x_i$) has zero mean and is decorrelated with the inputs x_i .

Linear regression: estimation

- Solution via matrix inversion

$$w_0\left(\sum_{i=1}^{n} 1\right) + w_1\left(\sum_{i=1}^{n} x_i\right) = \sum_{i=1}^{n} y_i$$
$$w_0\left(\sum_{i=1}^{n} x_i\right) + w_1\left(\sum_{i=1}^{n} x_i^2\right) = \sum_{i=1}^{n} y_i x_i$$

Or $\hat{w} = \Phi^{-1} b$, where

$$\Phi = \begin{bmatrix} \sum_{i=1}^{n} 1 & \sum_{i=1}^{n} x_i \\ \sum_{i=1}^{n} x_i & \sum_{i=1}^{n} x_i^2 \end{bmatrix}, \quad b = \begin{bmatrix} \sum_{i=1}^{n} y_i \\ \sum_{i=1}^{n} y_i x_i \end{bmatrix}$$

Linear regression

- In a matrix notation, we minimize:

$$\frac{1}{2}\left\| \begin{bmatrix} y_1 \\ \vdots \\ y_n \end{bmatrix} - \begin{bmatrix} 1 & x_1 \\ \cdots & \cdots \\ 1 & x_n \end{bmatrix} \begin{bmatrix} w_0 \\ w_1 \end{bmatrix} \right\|^2 \quad \text{or} \quad \frac{1}{2}\|y - Xw\|^2$$

- By setting the derivatives to zero, we get

$$X^T y - X^T X w = 0 \quad \Rightarrow \quad \hat{w} = \underbrace{(X^T X)}_{\Phi}^{-1} \underbrace{X^T y}_{b}$$

Note: the solution is a linear function of the outputs *y*

Linear Regression: Calculation Exercise

Given the five data points in Table 2, compute the optimal w_0 and w_1 in the linear regression function:

$$y = w_0 + w_1 x$$

	p1	p2	p3	p4	p5
(x, y)	(2, 3)	(4, 4)	(4, 6)	(6, 5)	(6, 7)

Solution

Given: $(x_i, y_i) = (2,3), (4,4), (4,6), (6,5), (6,7)$

Model: $\hat{y}_i = w_0 + w_1 x_i$

We aim to minimize $J(w) = \frac{1}{2}\|y - Xw\|^2$

Matrix form

$$X = \begin{bmatrix} 1 & 2 \\ 1 & 4 \\ 1 & 4 \\ 1 & 6 \\ 1 & 6 \end{bmatrix} \quad y = \begin{bmatrix} 3 \\ 4 \\ 6 \\ 5 \\ 7 \end{bmatrix} \quad w = \begin{bmatrix} w_0 \\ w_1 \end{bmatrix}$$

Products

$$X^T X = \begin{bmatrix} 5 & 22 \\ 22 & 108 \end{bmatrix} \quad X^T y = \begin{bmatrix} 25 \\ 118 \end{bmatrix}$$

$$X^T(Xw - y) = 0 \;\Rightarrow\; X^T X \hat{w} = X^T y$$

Solution

Using the normal equation: $\hat{w} = (X^T X)^{-1} X^T y$

1. Determinant and inverse

$$\det(X^T X) = 5 \cdot 108 - 22^2 = 56$$

$$(X^T X)^{-1} = 1/56 \begin{bmatrix} 108 & -22 \\ -22 & 5 \end{bmatrix}$$

2. Multiply by $X^T y$

$$\hat{w} = (X^T X)^{-1} X^T y = 1/56 \begin{bmatrix} 108 & -22 \\ -22 & 5 \end{bmatrix} \begin{bmatrix} 25 \\ 118 \end{bmatrix}$$

Component w_0

$$w_0 = (108 \cdot 25 - 22 \cdot 118)/56 = 104/56 = 13/7 \approx 1.857$$

Component w_1

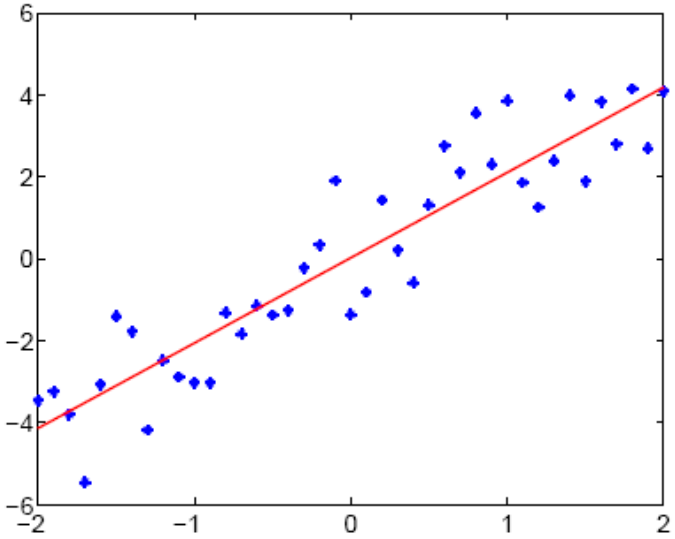
$$w_1 = (-22 \cdot 25 + 5 \cdot 118)/56 = 40/56 = 5/7 \approx 0.714$$

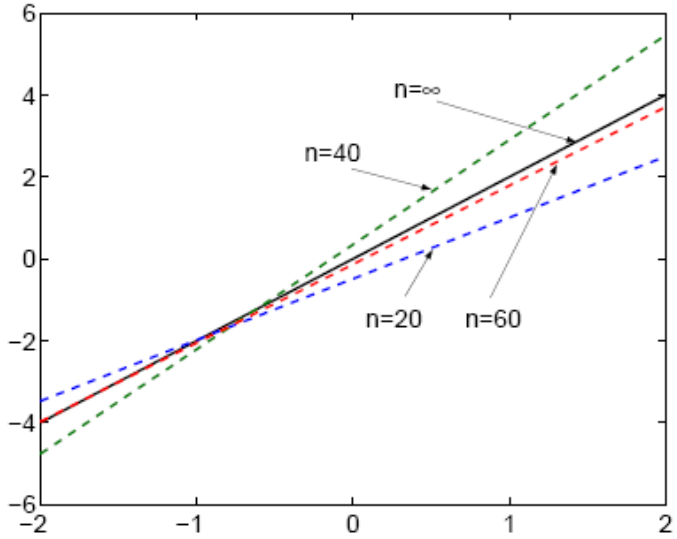
Final least-squares line

$$\hat{y} = w_0 + w_1 x = 13/7 + (5/7)x \approx 1.857 + 0.714x$$

Linear regression: generalization

- We'd like to understand how our linear predictions "improve" as a function of the number of training examples $\{(x_1; y_1); \ldots\ldots; (x_n; y_n)\}$





- We assume that there is a systematic relation between *x* and *y*: each training example (*x*; *y*) is an independent sample from a fixed but unknown distribution *P*

Linear Regression-extensionPolynomial Regression

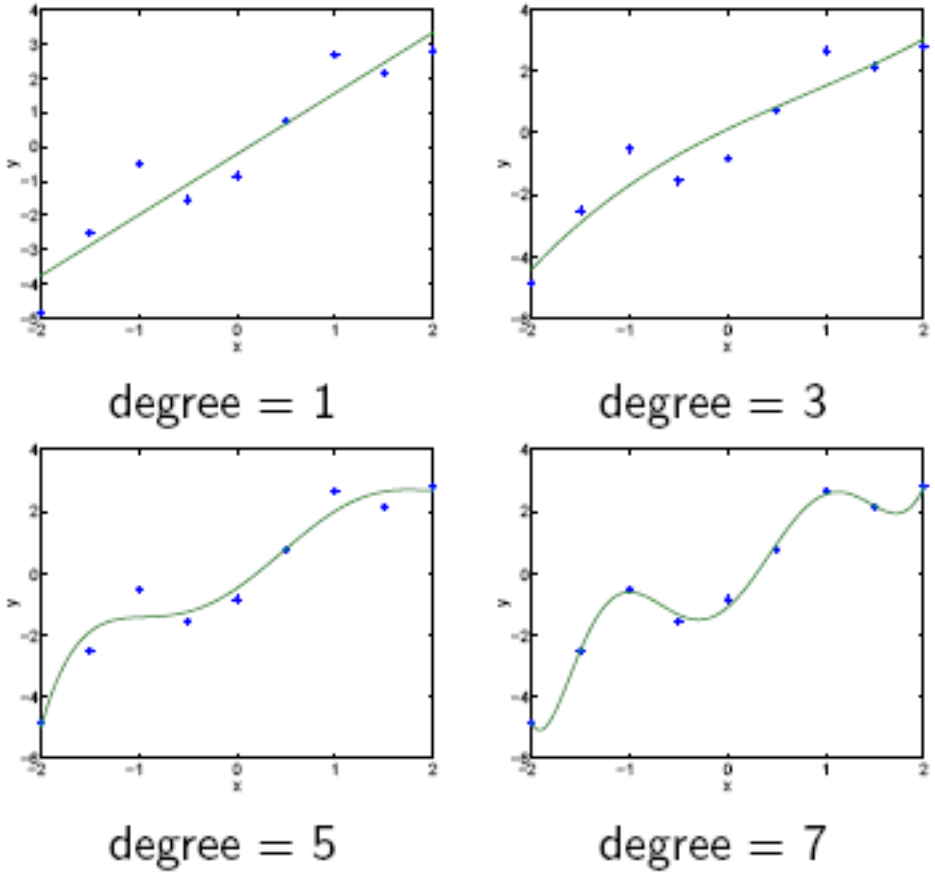
- Our previous results generalize to models that are linear in the parameters w
  - Simple linear prediction f : R→R

$$f(x\,;w) = w_0 + w_1 x$$

  - m^{th} order polynomial prediction f : R→R

$$f(x\,;w) = w_0 + w_1 x + \ldots + w_{m-1} x^{m-1} + w_m x^m$$

Polynomial Regression: Example



Linear regression: generalization

- Training examples { (x_1; y_1); …; (x_n; y_n) }
- Test examples {(x_{n+1}; y_{n+1}); …; (x_{n+N}; y_{n+N})}
  - $\hat{w}$ is the parameter estimated from the training examples.
- Types of errors:

$$\text{Mean training error} = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{w}_0 - \hat{w}_1 x_i)^2$$

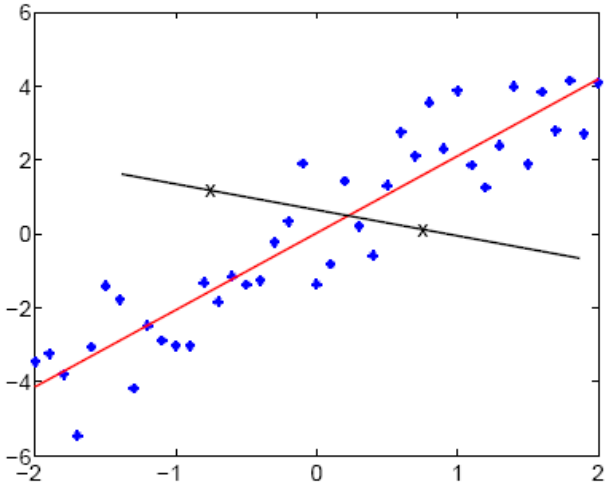
$$\text{Mean test error} = \frac{1}{N}\sum_{i=n+1}^{n+N}(y_i - \hat{w}_0 - \hat{w}_1 x_i)^2$$

$$\text{"Generalization" error} = E_{(x,y)\square P}\{(y - \hat{w}_0 - \hat{w}_1 x)^2\}$$

Note: $\hat{w}_0$ and $\hat{w}_1$ are themselves random variables as they depend on the training set.

Overfitting

- With too few training examples our linear regression model may achieve zero training error but it has a large generalization error



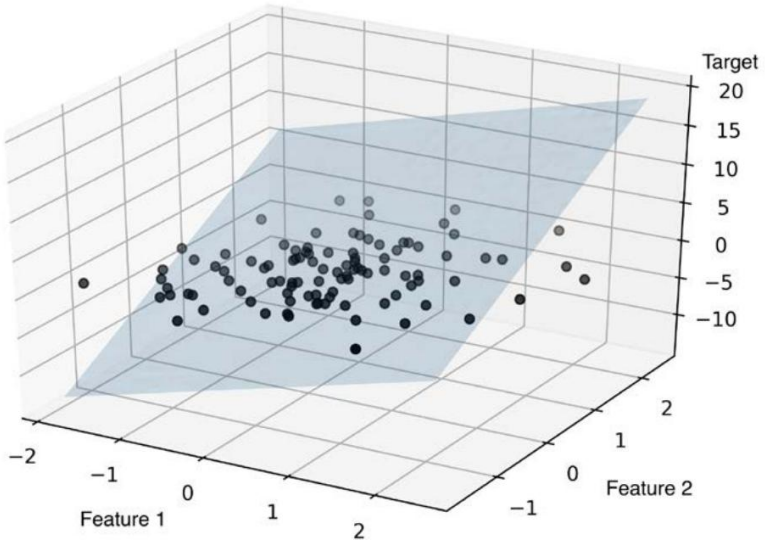
- When the training error no longer bears any relation to the generalization error the model overtfits the data.

Linear Regression-extension

- Generalize to multiple explanatory variables

$$y = w_0 x_0 + w_1 x_1 + \cdots + w_m x_m = \sum_{i=0}^{m} w_i x_i = w^T x$$

- Here, w_0 is the y axis intercept with $x_0 = 1$



Linear regression for 2-dim features

LOESS robust regression

Regular regression

- Assume y is generated using the following recipe:

$$y = w^T x + N(0, \sigma^2)$$

- Find the optimal w

LOESS

- Assume y is generated using the following recipe:

  With probability p:

$$y = w^T x + N(0, \sigma^2)$$

  With probability $1 - p$:

$$y \sim N(\mu, \sigma_{\text{huge}}^2)$$

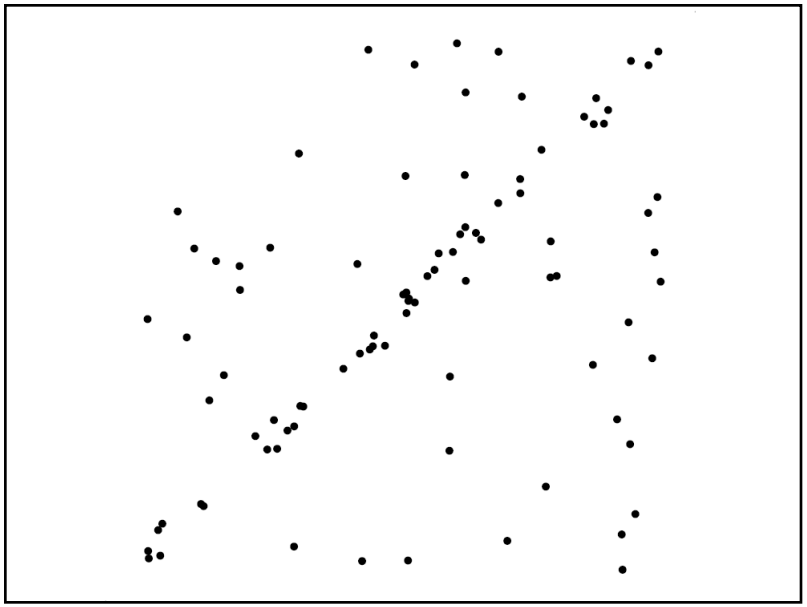
- Find the optimal w, $p, \mu, \sigma_{\text{huge}}$
  - Using EM algorithm

https://www.statsmodels.org/dev/generated/statsmodels.nonparametric.smoothers_lowess.lowess.html

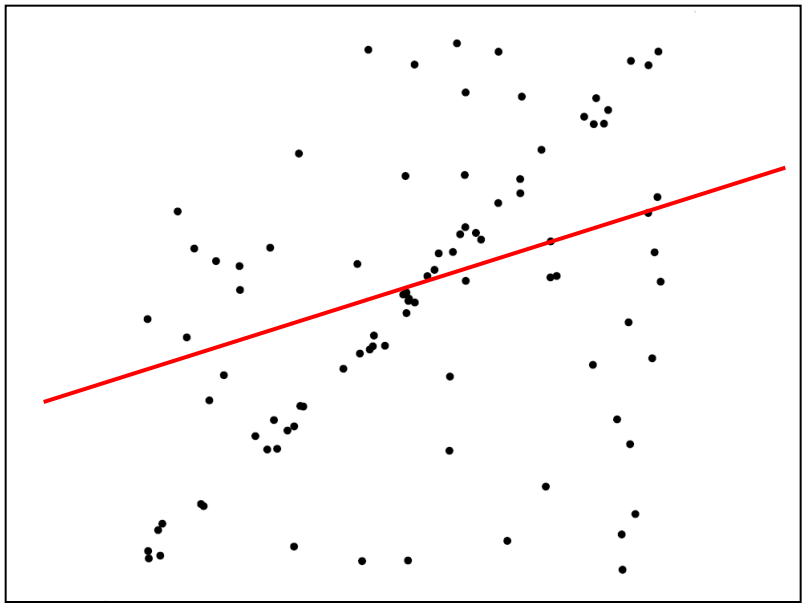
RANSAC

- RANdom SAmple Consensus (RANSAC): a very general framework for model fitting in the presence of a significant portion of outliers
- Follow the idea of LOESS to model the outlier explicitly, but in another way
- Outline
  - Choose a small subset of points uniformly at random
  - Fit a model to that subset
  - Find all remaining points that are “close” to the model and reject the rest as outliers
  - Do this many times and choose the best model

RANSAC for linear regression



RANSAC for linear regression

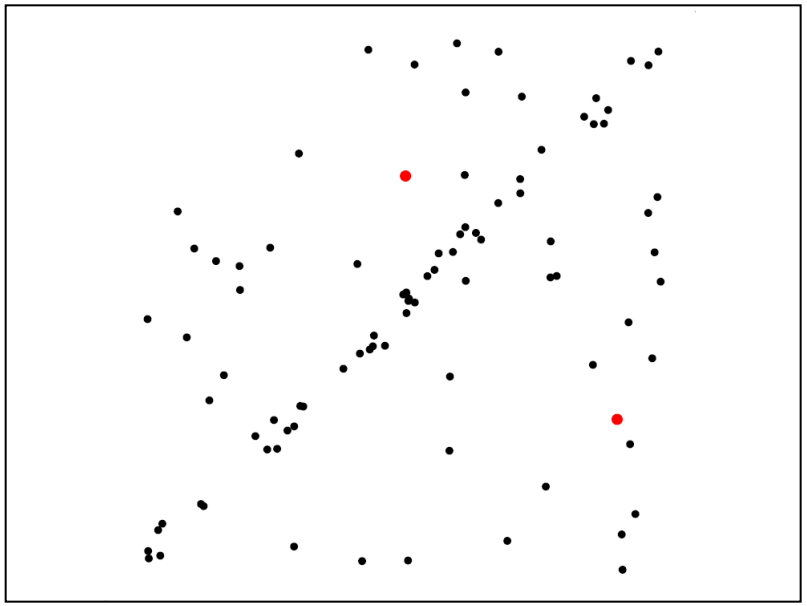


Least-squares fit

RANSAC for line fitting

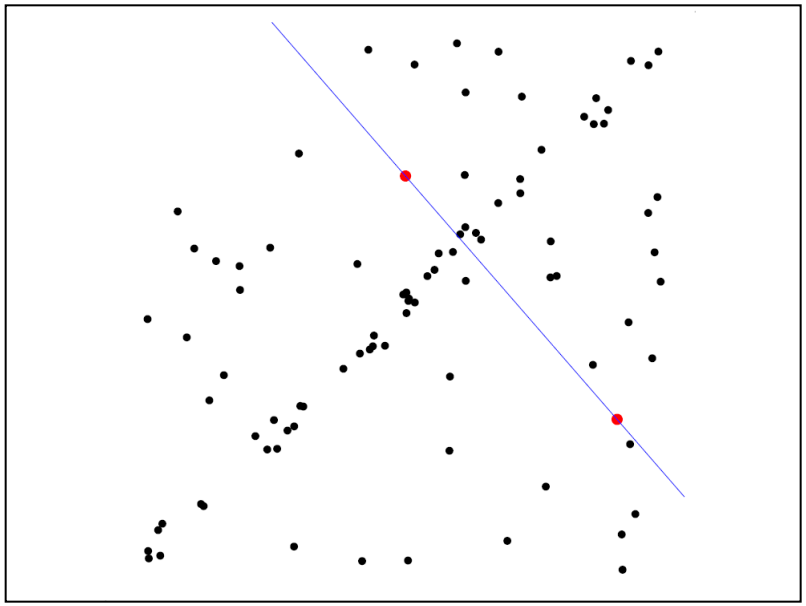
1. Repeat N times:
   - Draw s points uniformly at random
   - **Fit line** to these s points
   - Find inliers to this line among the remaining points (i.e., points whose distance from the line is less than t)
2. Select the largest inlier set, reject all outliers
3. Re-compute the solution using the optimal inlier set

RANSAC for linear regression



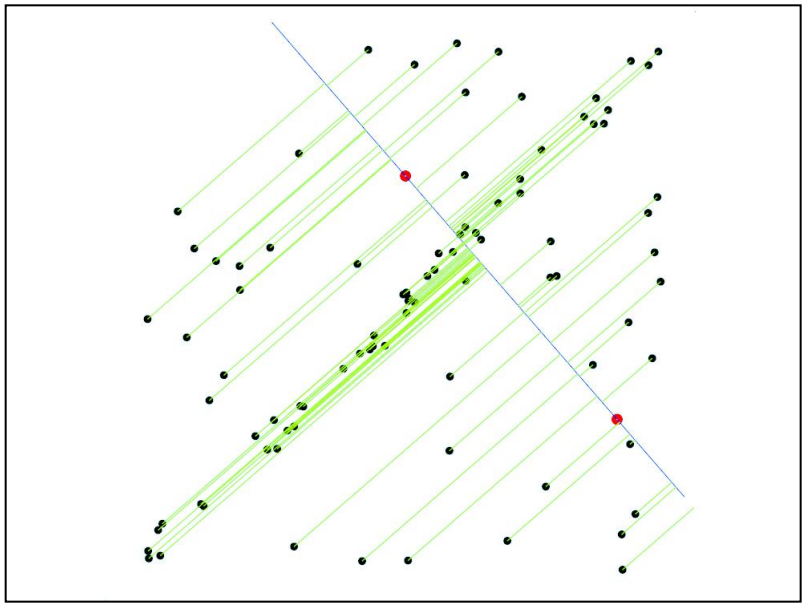
1. Randomly select minimal subset of points

RANSAC for linear regression



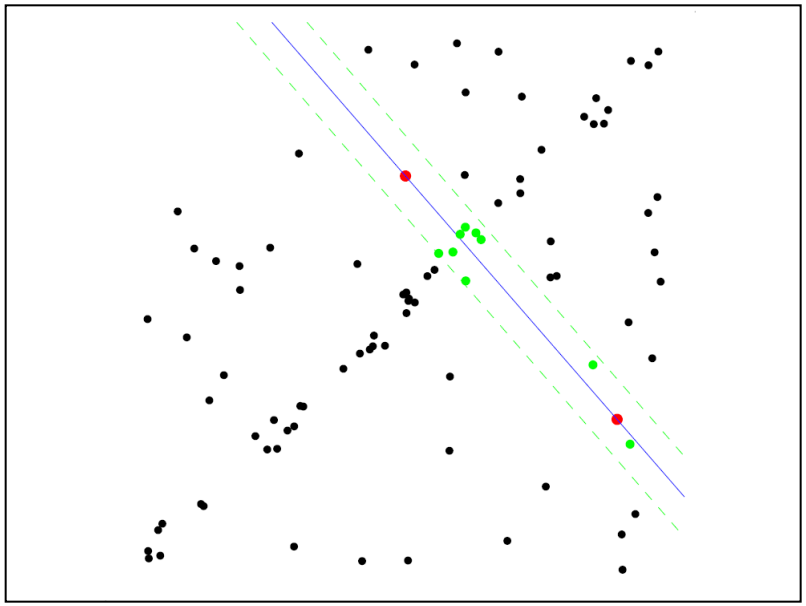
1. Randomly select minimal subset of points
2. Hypothesize a model

RANSAC for linear regression



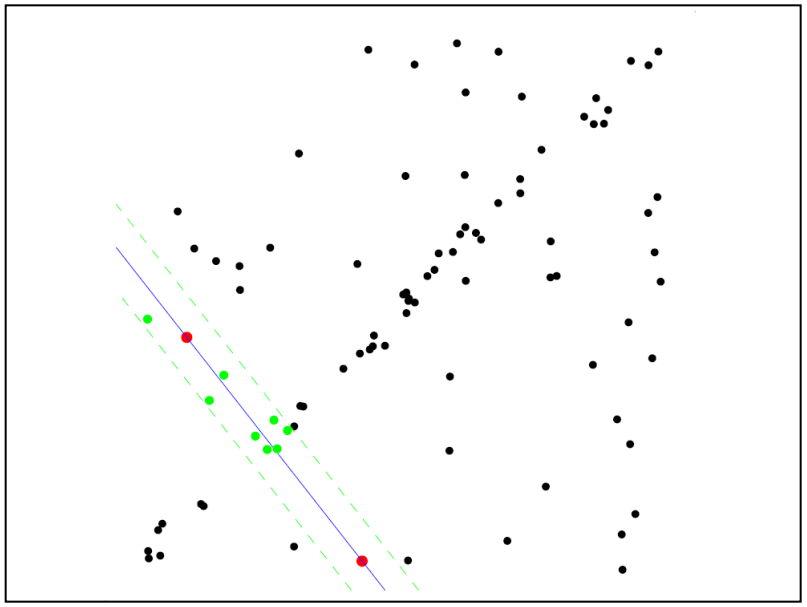
1. Randomly select minimal subset of points
2. Hypothesize a model
3. Compute error function

RANSAC for linear regression



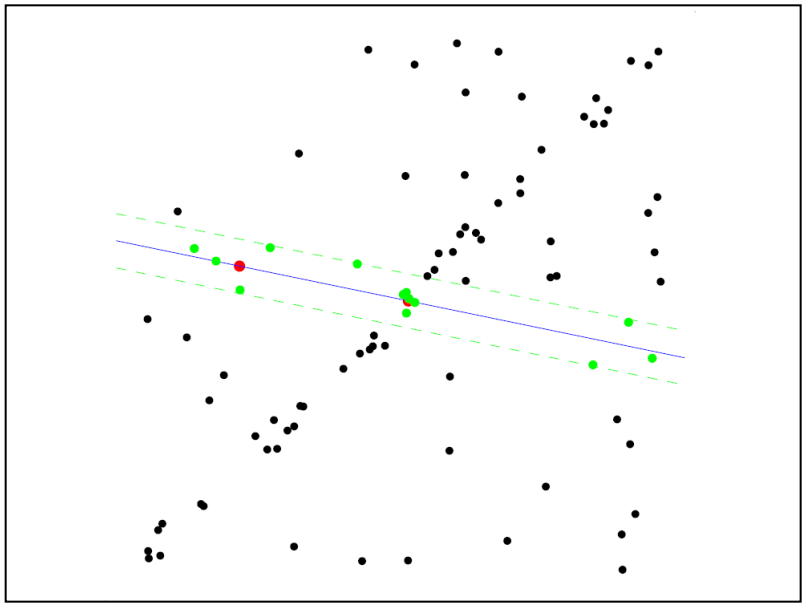
1. Randomly select minimal subset of points
2. Hypothesize a model
3. Compute error function
4. Select points consistent with model

RANSAC for linear regression



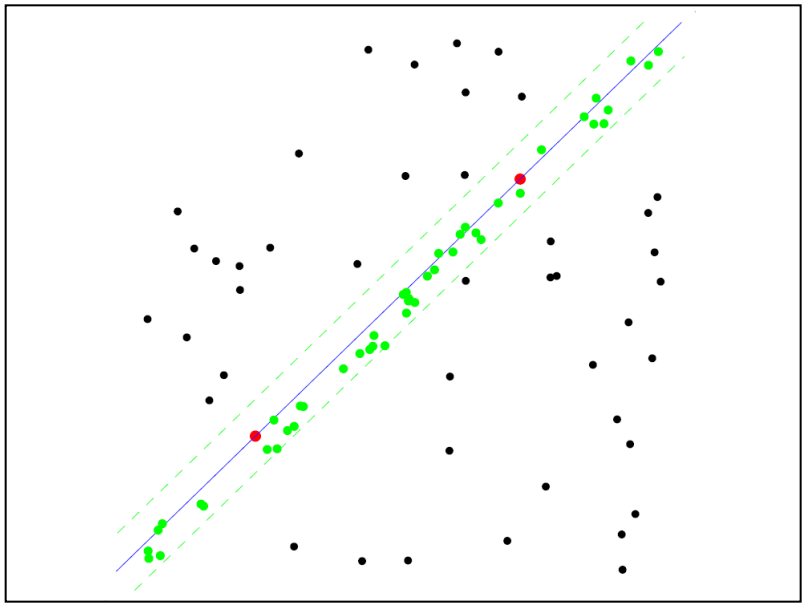
1. Randomly select minimal subset of points
2. Hypothesize a model
3. Compute error function
4. Select points consistent with model
5. Repeat *hypothesize-and-verify* loop

RANSAC for linear regression



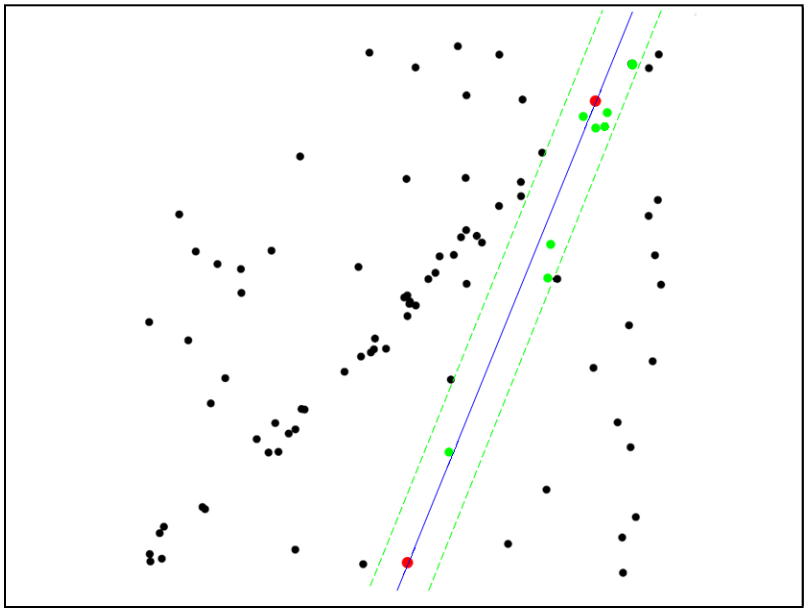
1. Randomly select minimal subset of points
2. Hypothesize a model
3. Compute error function
4. Select points consistent with model
5. Repeat *hypothesize-and-verify* loop

RANSAC for linear regression



1. Randomly select minimal subset of points
2. Hypothesize a model
3. Compute error function
4. Select points consistent with model
5. Repeat *hypothesize-and-verify* loop

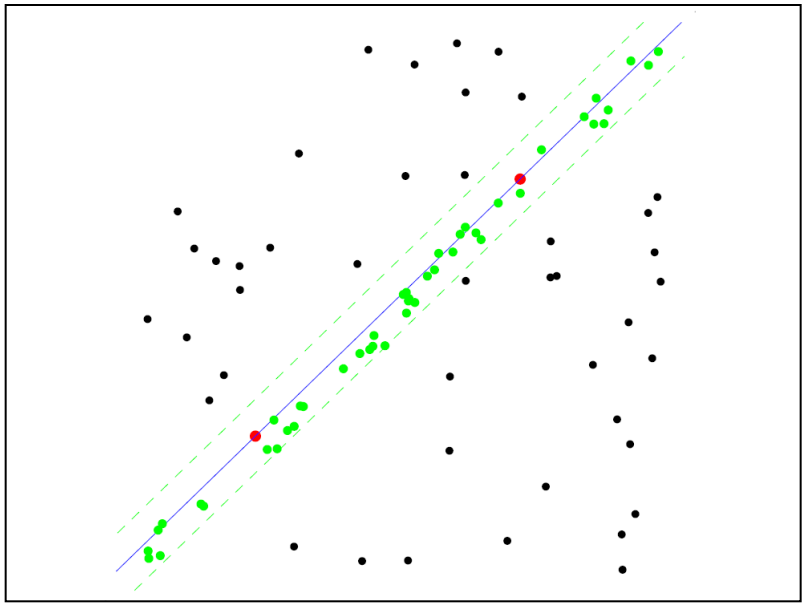
RANSAC for linear regression



1. Randomly select minimal subset of points
2. Hypothesize a model
3. Compute error function
4. Select points consistent with model
5. Repeat *hypothesize-and-verify* loop

RANSAC for linear regression

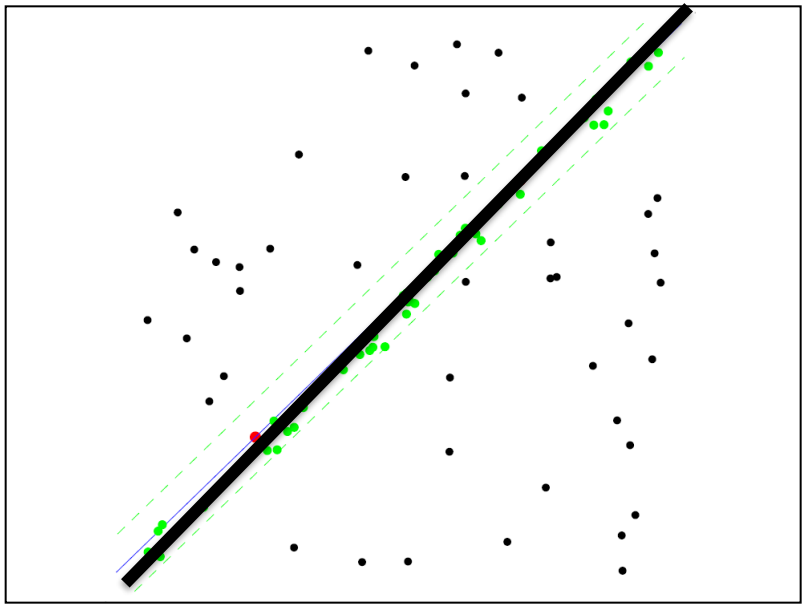
The hypothesis with the largest inliner set



1. Randomly select minimal subset of points
2. Hypothesize a model
3. Compute error function
4. Select points consistent with model
5. Repeat hypothesize-and-verify loop

RANSAC for linear regression

Recompute the linear fitting



1. Randomly select minimal subset of points
2. Hypothesize a model
3. Compute error function
4. Select points consistent with model
5. Repeat hypothesize-and-verify loop

Outline of Lecture

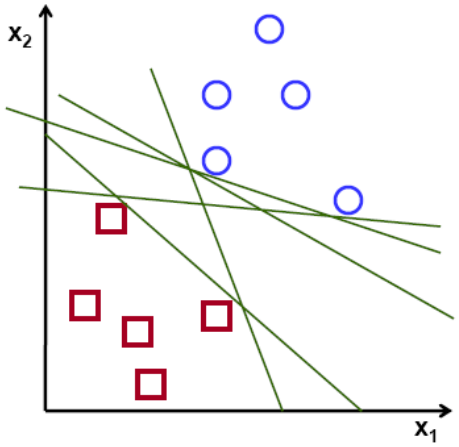
- Linear regression
  - Estimation criterion
  - Least squares solution
  - Generalization and overfitting
  - Extensions
- Support Vector Machine
  - Maximum Margin Hyperplane
  - The Lagrangian Dual Problem
  - The non-separable case & Non-linear SVMs and kernel methods
  - Optimization techniques
- Cross-Validation
- Performance Evaluation

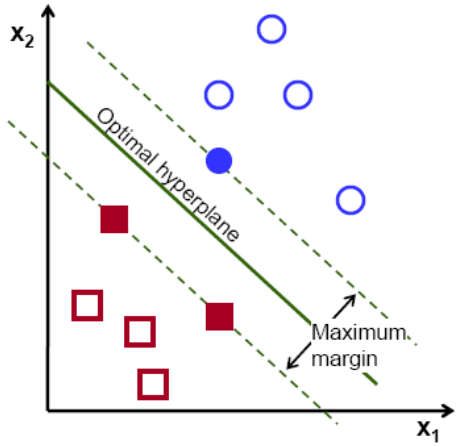
Support Vector Machine Empirical Risk Minimization

- Empirical Risk Minimization (ERM)
  - **A formal term for a simple concept: find the function *f*(x) that minimizes the average risk on the training set**
  - **Minimizing the empirical risk is not a bad thing to do, provided that sufficient training data is available, since the law of large numbers ensures that the empirical risk will asymptotically converge to the expected risk for n→∞**
  - **However, for small samples, one cannot guarantee that ERM will also minimize the expected risk. This is the all too familiar issue of generalization**
- How do we avoid overfitting?
  - **By controlling model complexity. Intuitively, we should prefer the simplest model that explains the data**

Optimal separating hyperplanes

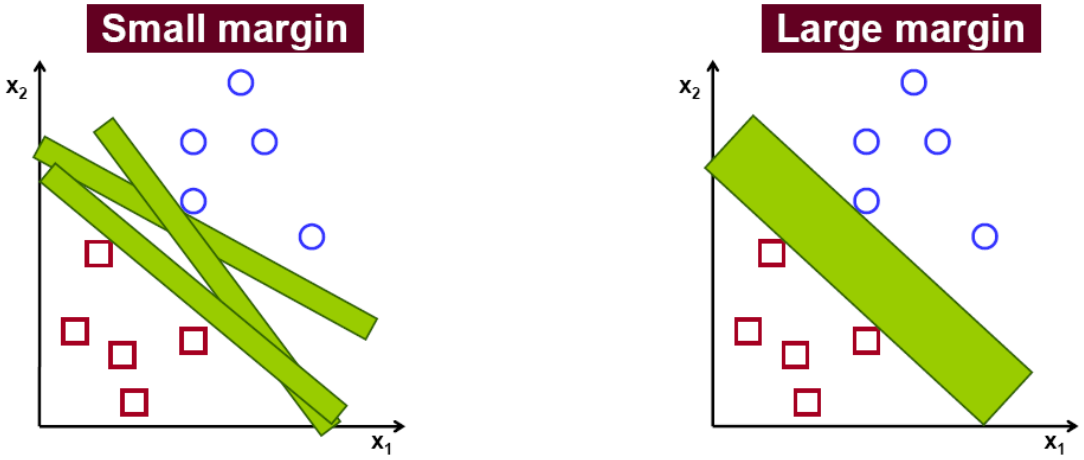
- Consider the problem of finding a separating hyperplane for a linearly separable dataset $\{(x_1,y_1),(x_2,y_2),\ldots,(x_N,y_N)\}$, $x \in R^D$, $y \in \{-1,+1\}$
  - Which of the infinite hyperplanes should we choose?
    - Intuitively, a hyperplane that passes too close to the training examples will be sensitive to noise and, therefore, less likely to generalize well for data outside the training set
    - Instead, it seems reasonable to expect that a hyperplane that is farthest from all training examples will have better generalization capabilities
- Therefore, the optimal separating hyperplane will be the one with the largest margin, *which is defined as the minimum distance of an example to the decision surface*





Optimal separating hyperplanes

- To further understand the relationship between margin and capacity, consider the two separating hyperplanes depicted below
  - A “skinny” one (small margin), which will be able to adopt many orientations
  - A “fat” one (large margin), which will have limited flexibility
- A larger margin necessarily results in lower capacity
  - We normally think of complexity as being a function of the number of parameters
    - Instead, Statistical Learning Theory tells us that if the margin is sufficiently large, the complexity of the function will be low even if the dimensionality is very high!

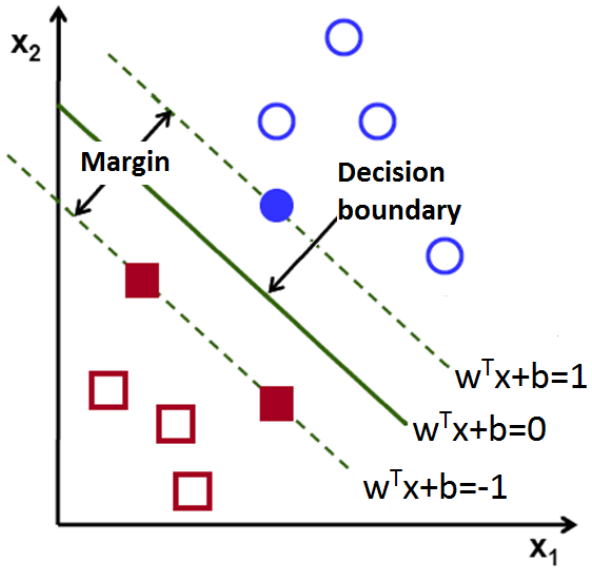


Optimal separating hyperplanes

- Pick an arbirary point x_1 to lie on line $w^T x + b = -1$. Then, the closest point on line $w^T x + b = 1$ to x_1 is the point x_2, where

$$x_2 = x_1 + \lambda w$$

  - since the closest point will always lie on the perpendicular;
  - the vector w is perpendicular to both lines
- Using this formulation, λw will be the line segment connecting x_1 and x_2, and thus, $\lambda \|w\|$, the margin between x_1 and x_2, is the shortest distance between the two lines/boundaries.



Optimal separating hyperplanes

Solving for λ :

- First we have

$$w^T x_2 + b = 1 \quad \text{where} \quad x_2 = x_1 + \lambda w$$

- Then we can get

$$w^T x_1 + b + \lambda w^T w = 1 \quad \text{where} \quad w^T x_1 + b = -1$$

- That is

$$\lambda w^T w = 2 \qquad \text{so} \quad \lambda = \frac{2}{\|w\|^2}$$

- Finally, the margin $\lambda \|w\|$ equals $\dfrac{2}{\|w\|}$

Optimal separating hyperplanes

- Therefore, the problem of maximizing the margin is equivalent to

$$\text{minimize} \qquad \mathrm{J(w)}=\frac{1}{2}\|w\|^2$$

$$\text{subject to} \qquad y_i(w^T x_i + b) \geq 1 \quad \forall i$$

  - Notice that *J(w)* is a quadratic function, which means that there exists a single global minimum and no local minima
- To solve this problem, we will use classical Lagrangian optimization techniques

Lagrangian Method

- A general non-linear constraint optimization problem (**OP**) may be considered as minimizing $f(w)$ subject to constraints

$$g_i(w) \leq 0, \quad i = 1, 2, \ \ldots, m$$
$$h_j(w) = 0, \quad j = 1, 2, \ \ldots, p$$

- where f, g_i and h_j are functions of parameters, and

$$w = (w_1, w_2, \ \ldots, w_n) \in \mathbb{R}^D$$

- Function f is referred to as the *cost* function. Functions g and h are the inequality and equality constraint functions, respectively.

Lagrangian Method

- Lagrange multiplier specifies to optimize an augmented cost function L_P:

$$L_p(w, a, \lambda) = f(w) + \sum_{i=1}^{m} a_i g_i(w) + \sum_{j=1}^{p} \lambda_j h_j(w)$$

  - The non-negative constants a_i and λ_j are known as Lagrange multipliers.
  - The above optimization function can be optimized by equating the derivatives with respect to parameters and Lagrange multipliers and equating to zero.
- In addition, the inequality constraints satisfies the following condition (known as Karush-Kuhn-Tucker (KKT) condition):

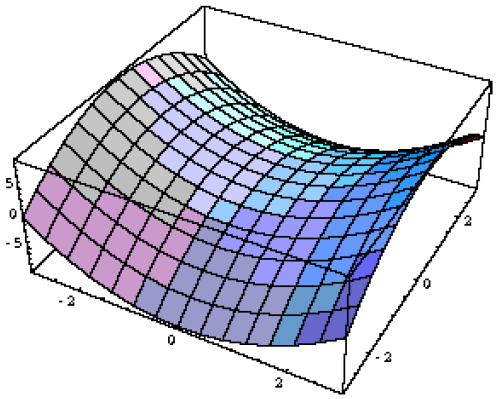
$$a_i g_i(w) = 0, \quad \text{for all } i = 1, 2, \ \ldots \ , m$$

The Lagrangian dual problem

- Constrained minimization of $J(w)=1/2||w||^2$ is solved by introducing the Lagrangian

$$L_p(w,b,a)=\frac{1}{2}\|w\|^2+\sum_{i=1}^{N}a_i\left[1-y_i(w^T x_i+b)\right]$$

  - which yields an unconstrained optimization problem that is solved by:
    - minimizing L_P with respect to the primal variables w and b, and
    - maximizing L_P with respect to the dual variables $\alpha_i \geq 0$ (the Lagrange multipliers)
  - Thus, the optimum is defined by a saddle point (see below for illustration)
- This is known as the Lagrangian primal problem



A saddle point

The Lagrangian dual problem

- To simplify the primal problem, we eliminate the primal variables (*w*,*b*) using the first Kuhn-Tucker condition *∂Lp/∂w=0* and *∂Lp/∂b=0*
  - Differentiating $L_p(w,b,a)$ with respect to *w* and *b*, and setting to zero yields

$$\frac{\partial L_p(w,b,a)}{\partial w} = 0 \Rightarrow w = \sum_{i=1}^{N} a_i y_i x_i$$

$$\frac{\partial L_p(w,b,a)}{\partial b} = 0 \Rightarrow \sum_{i=1}^{N} a_i y_i = 0$$

  - Expansion of L_P yields

$$L_p(w \mid b,a) = \frac{1}{2} w^T w - \sum_{i=1}^{N} a_i y_i w^T x_i - b\sum_{i=1}^{N} a_i y_i + \sum_{i=1}^{N} a_i$$

  - Using the optimality condition ∂L_p/∂w=0, the first term in L_P can be expressed as

$$w^T w = w^T \sum_{i=1}^{N} a_i y_i x_i = \sum_{i=1}^{N} a_i y_i w^T x_i = \sum_{i=1}^{N} a_i y_i \left(\sum_{j=1}^{N} a_j y_j x_j \right)^T x_i = \sum_{i=1}^{N}\sum_{j=1}^{N} a_i a_j y_i y_j x_i^T x_j$$

  - The second term in L_P can be expressed in the same way
  - The third term in L_P is zero by using the optimality condition *∂Lp/∂b*=0

The Lagrangian dual problem

– Merging these expressions together, we maximize the following objective function

$$L_D(a) = \sum_{i=1}^{N} a_i - \frac{1}{2} \sum_{i=1}^{N} \sum_{j=1}^{N} a_i a_j y_i y_j x_i^T x_j$$

Subject to the (simpler) constraints $a_i \geq 0$ and $\sum_{i=1}^{N} a_i y_i = 0$

- This is known as the Lagrangian dual problem

- How to classify any test data

$$f(x^{test}) = w^T x^{test} + b = \sum_{i=1}^{N} a_i y_i x_i^T x^{test} + b$$

The Lagrangian dual problem

We maximize the following objective function

$$L_D(a) = \sum_{i=1}^{N} a_i - \frac{1}{2}\sum_{i=1}^{N}\sum_{j=1}^{N} a_i a_j y_i y_j x_i^T x_j$$

Subject to the (simpler) constraints $a_i \geq 0$ and $\sum_{i=1}^{N} a_i y_i = 0$

- Comments
  - We have transformed the problem of finding a saddle point for $L_P(w,b)$ into the easier one of maximizing $L_D(a)$
    - Notice that $L_D(a)$ depends on the Lagrange multipliers a, not on (w,b)
  - The primal problem scales with dimensionality (w has one coefficient for each dimension), whereas the dual problem scales with the amount of training data (there is one Lagrange multiplier per example)
  - Moreover, in $L_D(a)$ the training data appears only as dot products $x_i^T x_j$
    - As we will see, this property can be cleverly exploited to perform the classification in a higher (e.g., infinite) dimensional space

Support Vectors

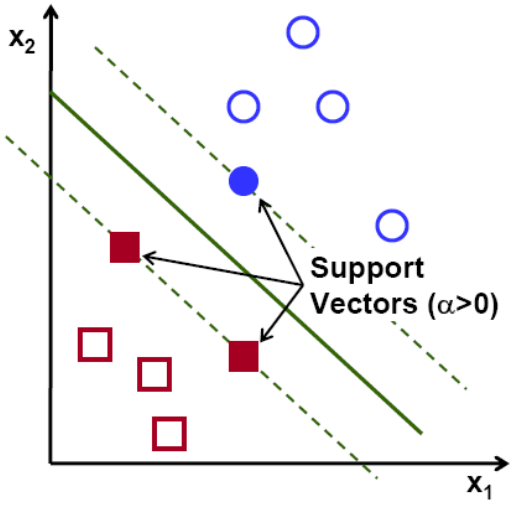
- The Karush-Kuhn-Tucker (KKT) condition states that, for every point in the training set, the following equality must hold

$$a_i \left[y_i (w^T x_i + b) - 1 \right] = 0 \qquad \forall i = 1 \ldots N$$

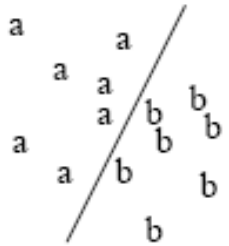
  - Therefore, for each example, either $a_i = 0$ or $y_i (w^T x_i + b) = 1$ must hold
  - Those points for which $a_i > 0$ must then lie on one of the two hyperplanes that define the largest margin (only at these hyperplanes the term $y_i (w^T x_i + b) - 1$ becomes zero)
    - These points are known as the Support Vectors
  - All the other points must have $a_i = 0$
  - Note that only the support vectors contribute to defining the optimal hyperplane

$$f(x^{test}) = w^T x^{test} + b = \sum_{i=1}^{N} a_i y_i x_i^T x^{test} + b$$

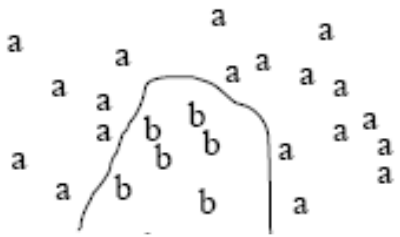
Therefore, the complete dataset could be replaced by only support vectors, and the separating hyperplane is the same



Linearly separable vs. Nonlinearly separable



Linearly separable

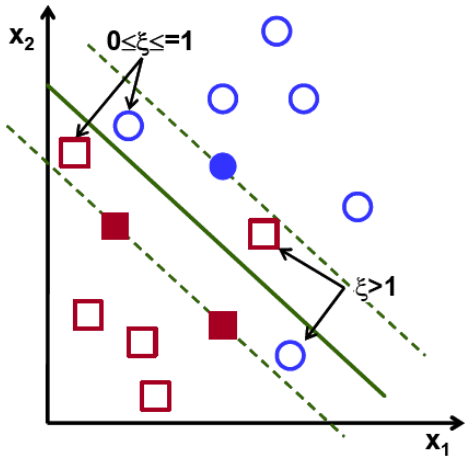

a a a a a a a a

The non-separable case

- The previous part focused on problems that were linearly separable
  - In this lecture we will see how SVMs can be modified to handle datasets that are not linearly separable
- The solution for the non-separable case is to introduce slack variables ξ_i that relax the constraints of the canonical hyperplane equation

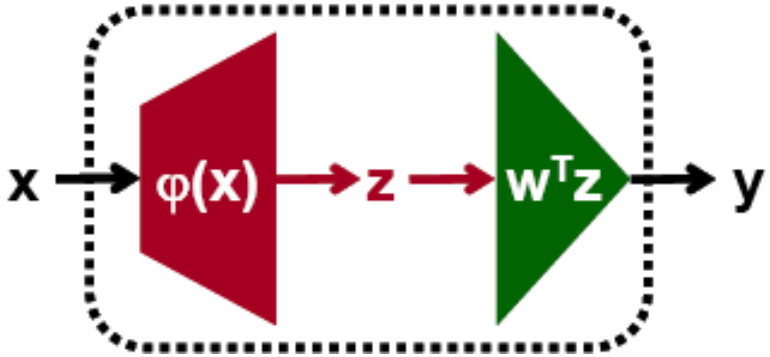
$$y_i(w^T x_i + b) \geq 1 - \xi_i, \qquad \forall i = 1, \ldots, N$$

- The slack variables measure deviation from the ideal condition
  - For $0 \leq \xi_i \leq 1$, the data point falls on the right side of the separating hyperplane but within the region of maximum margin
  - For $\xi_i > 1$, the data point falls on the wrong side of the separating hyperplane

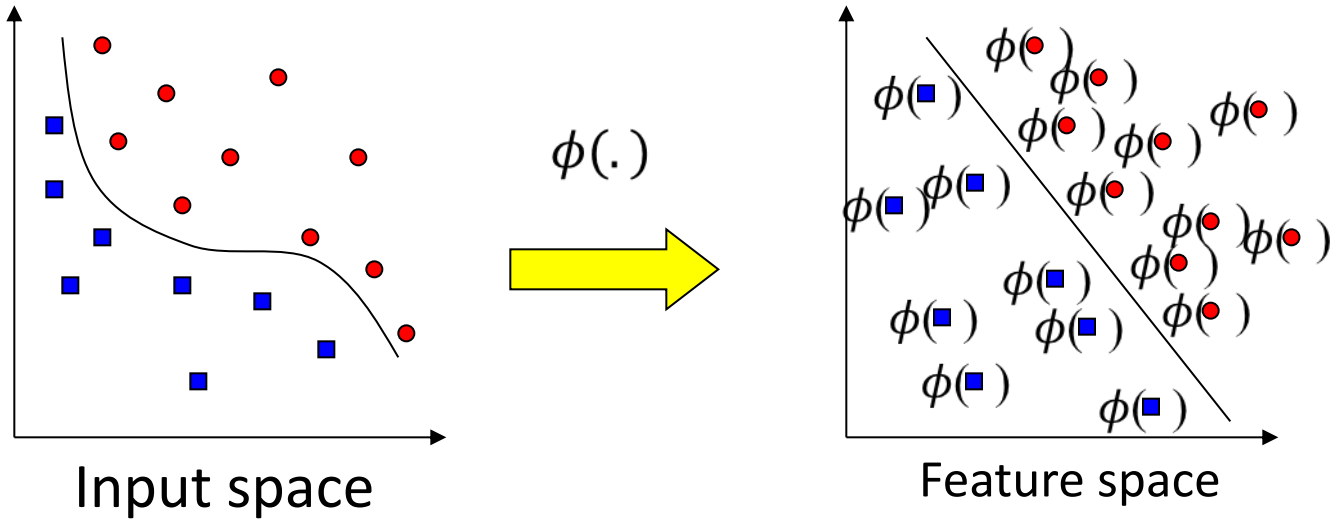


Non-linear SVMs

- The power of SVMs resides in the fact that they represent a robust and efficient implementation of the principle in Cover's theorem on the separability of patterns
  - *"A complex pattern-classification problem cast in a high-dimensional space nonlinearly is more likely to be linearly separable than in a low-dimensional space"*
- Based on this principle, SVMs operate in two stages
  - Perform a non-linear mapping of the feature vector *x* onto a high-dimensional space that is hidden from the inputs or the outputs
  - Construct an optimal separating hyperplane in the high-dimensional space



Transforming the data



Note: feature space is of higher dimension than the input space in practice

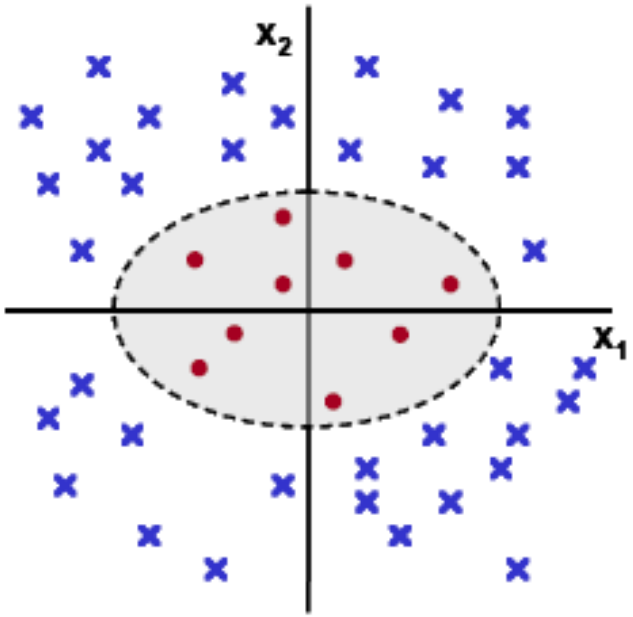
- Computation in the feature space can be costly because it is high dimensional (could be infinite-dimensional)
- The kernel trick comes to rescue

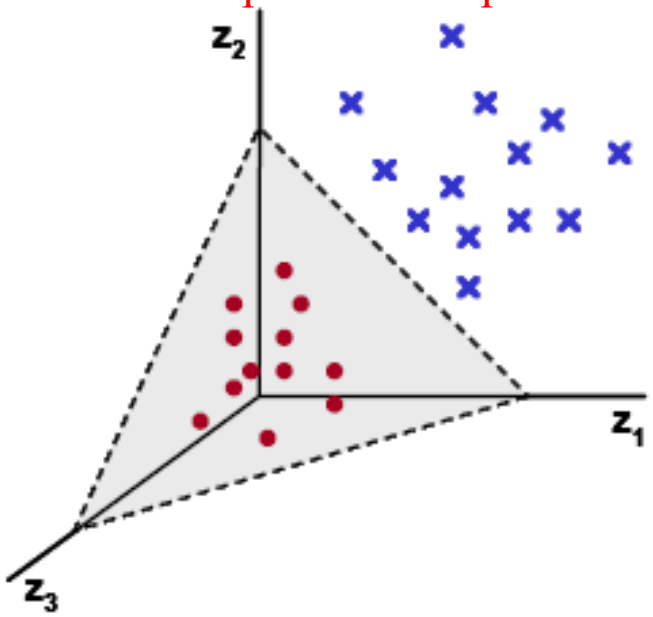
Non-linear SVMs

$$\varphi : R^2 \rightarrow R^3 \quad \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} \rightarrow \begin{bmatrix} z_1 \\ z_2 \\ z_3 \end{bmatrix} = \begin{bmatrix} x_1^2 \\ \sqrt{2}x_1x_2 \\ x_2^2 \end{bmatrix}$$

$$\varphi(x_i) = \begin{pmatrix} x_{i,1}^2 \\ \sqrt{2}x_{i,1}x_{i,2} \\ x_{i,2}^2 \end{pmatrix}$$

One example mapping function from 2D space to 3D space





Implicit mappings: an example

- Consider a classification problem in R^2
  - Assume we choose a kernel function $K(x_i, x_j) = (x_i^T x_j)^2$
  - Our goal is to find a non-linear projection $\varphi(x)$ such that $(x_i^T x_j)^2 = \varphi(x_i)^T \varphi(x_j)$
  - Performing the expansion of $K(x_i, x_j)$

$$K(x_i, x_j) = (x_i^T x_j)^2 = \left(\begin{bmatrix} x_{i,1} & x_{i,2} \end{bmatrix} \begin{bmatrix} x_{j,1} \\ x_{j,2} \end{bmatrix} \right)^2$$

$$= (x_{i,1} x_{j,1} + x_{i,2} x_{j,2})^2 \quad \text{Inner product in 2D space}$$

$$= x_{i,1}^2 x_{j,1}^2 + 2 x_{i,1} x_{j,1} x_{i,2} x_{j,2} + x_{i,2}^2 x_{j,2}^2$$

$$= \begin{bmatrix} x_{i,1}^2 & \sqrt{2} x_{i,1} x_{i,2} & x_{i,2}^2 \end{bmatrix} \begin{bmatrix} x_{j,1}^2 \\ \sqrt{2} x_{j,1} x_{j,2} \\ x_{j,2}^2 \end{bmatrix} \quad \text{Inner product in 3D space}$$

  - where $x_{i,k}$ denotes the *k*-th coordinate of example x_i

  Notice that the inner product $\varphi(x_i)^T \varphi(x_j)$ can be computed in R^2 (the original space) by means of the kernel $(x_i^T x_j)^2$ without ever having to project onto R^3!

Example for feature mapping & kernels

- Consider an input $x = [x_1, x_2]$
- Suppose $\phi(\cdot)$ is given as

$$\phi\left(\begin{pmatrix} x_1 \\ x_2 \end{pmatrix}\right) = [1, \sqrt{2}x_1, \sqrt{2}x_2, x_1^2, x_2^2, \sqrt{2}x_1x_2]$$

- An inner product in the feature space is

$$\left\langle \phi\left(\begin{pmatrix} x_1 \\ x_2 \end{pmatrix}\right), \phi\left(\begin{pmatrix} x_1' \\ x_2' \end{pmatrix}\right) \right\rangle = \begin{array}{c} 1 + 2x_1x_1' + 2x_2x_2' + x_1^2x_1'^2 + x_2^2x_2'^2 + 2x_1x_2x_1'x_2' \\ 1 + 2(x_1x_1' + x_2x_2') + (x_1x_1' + x_2x_2')^2 \\ (x_1x_1' + x_2x_2' + 1)^2 \end{array}$$

- Thus, if we define the kernel function as $k(x, x') = \left(1 + x^T x'\right)^2$, there is not need to carry out $\phi(\cdot)$ explicitly

Kernel methods

- How do we compute the dual variables a_i in the implicit space?
  - Very simple: we use the same optimization problem as before, except for now we replace the dot product $\varphi(x_i)^T \varphi(x_j)$ by the kernel $K(x_i, x_j)$
- The Lagrangian dual problem for the non-linear SVM is simply

$$L_D(a) = \sum_{i=1}^{N} a_i - \frac{1}{2} \sum_{i=1}^{N} \sum_{j=1}^{N} a_i a_j y_i y_j K(x_i, x_j)$$

subject to the constraints $\begin{cases} \sum_{i=1}^{N} a_i y_i = 0 \\ 0 \le a_i \le C \quad i = 1 \ldots N \end{cases}$

More examples of kernel functions

- Linear kernel (as we've seen)

$$k(x, x') = x^T x'$$

- Polynomial kernel (we just saw)

$$k(x, x') = \left(1 + x^T x'\right)^d$$

where d=2, 3,.. To get the feature vectors we concatenate all d-th order polynomial terms of the components of x (weighted appropriately)

- Radial basis kernel

$$k(x, x') = \exp\left(-\frac{1}{2}\|x - x'\|^2\right)$$

- In this case, the feature space consists of functions and results in a non-parametric classifier

Optimization techniques

- SVMs involve the solution of a quadratic programming (QP) problem

$$J(\alpha) = 1^T \alpha - \frac{1}{2} \alpha^T H \alpha$$

$$\text{where } H_{ij} = y_i y_j K(x_i, x_j), \text{subject to constraints } \sum_{i=1}^{N} \alpha_i y_i = 0; 0 \le \alpha_i \le C, i = 1, \ldots, N$$

- Several commercial optimization libraries can be used to solve this dual QP problem
  - The use of these libraries is, however, limited to small- to medium-size problems
  - (1,000 examples) since the number of elements in the quadratic matrix *H* is equal to the square of the number of training examples
- A number of alternative optimization procedures have therefore been proposed by the SVM community
  - Chunking
  - Decomposition methods
  - Sequential Minimal Optimization

Fusion in SVM

- Early Fusion
  - Concatenates multiple feature vectors into a longer feature vector
  - Averages multiple kernels from different features to generate a single kernel

$$K^{Fuse} = K_1 + K_2,$$

where K_1 and K_2 are two kernels,

K^{Fuse} is the kernel after fusion

- Late Fusion
  - Scale normalization
  - Simple fusion techniques (i.e., MAX, MIN, AVERAGE and Weighted AVERAGE)
  - Example:

$$f^{Fuse} = \frac{0.5}{1+\exp(-f_1)} + \frac{0.5}{1+\exp(-f_2)}$$

where f_1 and f_2 are the scores (decision values) from two SVMs

f^{Fuse} is the score after fusion

References

- LIBSVM: http://www.csie.ntu.edu.tw/~cjlin/libsvm/
- Tutorial: A Tutorial on Support Vector Machines for Pattern Recognition, Christopher J.C. Burges

Outline of Lecture

- Linear regression
  - Estimation criterion
  - Least squares solution
  - Generalization and overfitting
  - Extensions
- Support Vector Machine
  - Maximum Margin Hyperplane
  - The Lagrangian Dual Problem
  - The non-separable case & Non-linear SVMs and kernel methods
  - Optimization techniques
- Cross-Validation
- Performance Evaluation

Cross-validation

- Almost invariably, all the pattern recognition techniques that we introduced have one or more free parameters
  - The bandwidth parameter and C in SVM

- Two issues arise at this point
  - **Model Selection**: How do we select the "optimal" parameter(s) for a given classification problem?
  - **Validation:**
    - Once we have chosen a model, how do we estimate its true error rate?
    - The true error rate is the classifier's error rate when tested on the ENTIRE POPULATION

Cross-validation

- If we had access to an unlimited number of examples, these questions would have a straightforward answer
  - Choose the model that provides the lowest error rate on the entire population
  - And, of course, that error rate is the true error rate

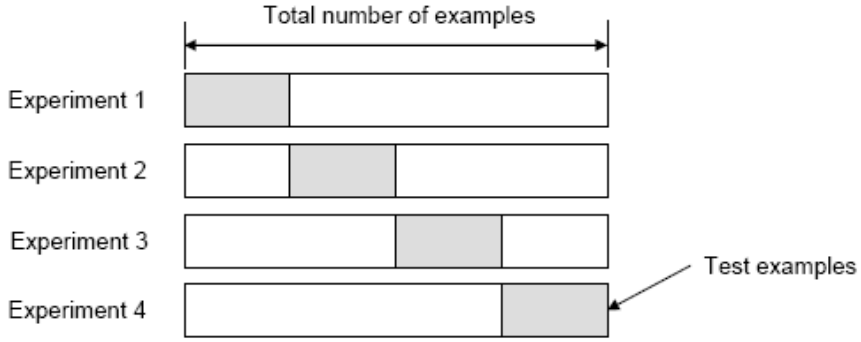
- However, in real applications only a finite set of examples is available
  - This number is usually smaller than we would hope for!
  - Why? Data collection is a very expensive process

Cross-validation

- One may be tempted to use the entire training data to select the "optimal" classifier, *then* estimate the error rate
- This naive approach has two fundamental problems
  - The final model will normally overfit the training data: it will not be able to generalize to new data.
    - The problem of overfitting is more pronounced with models that have a large number of parameters.
  - The error rate estimate will be overly optimistic (lower than the true error rate)
    - In fact, it is not uncommon to have 100% correct classification on training data
- The techniques presented in this lecture will allow you to make the best use of your (limited) data for
  - Training, Model selection, and Performance estimation

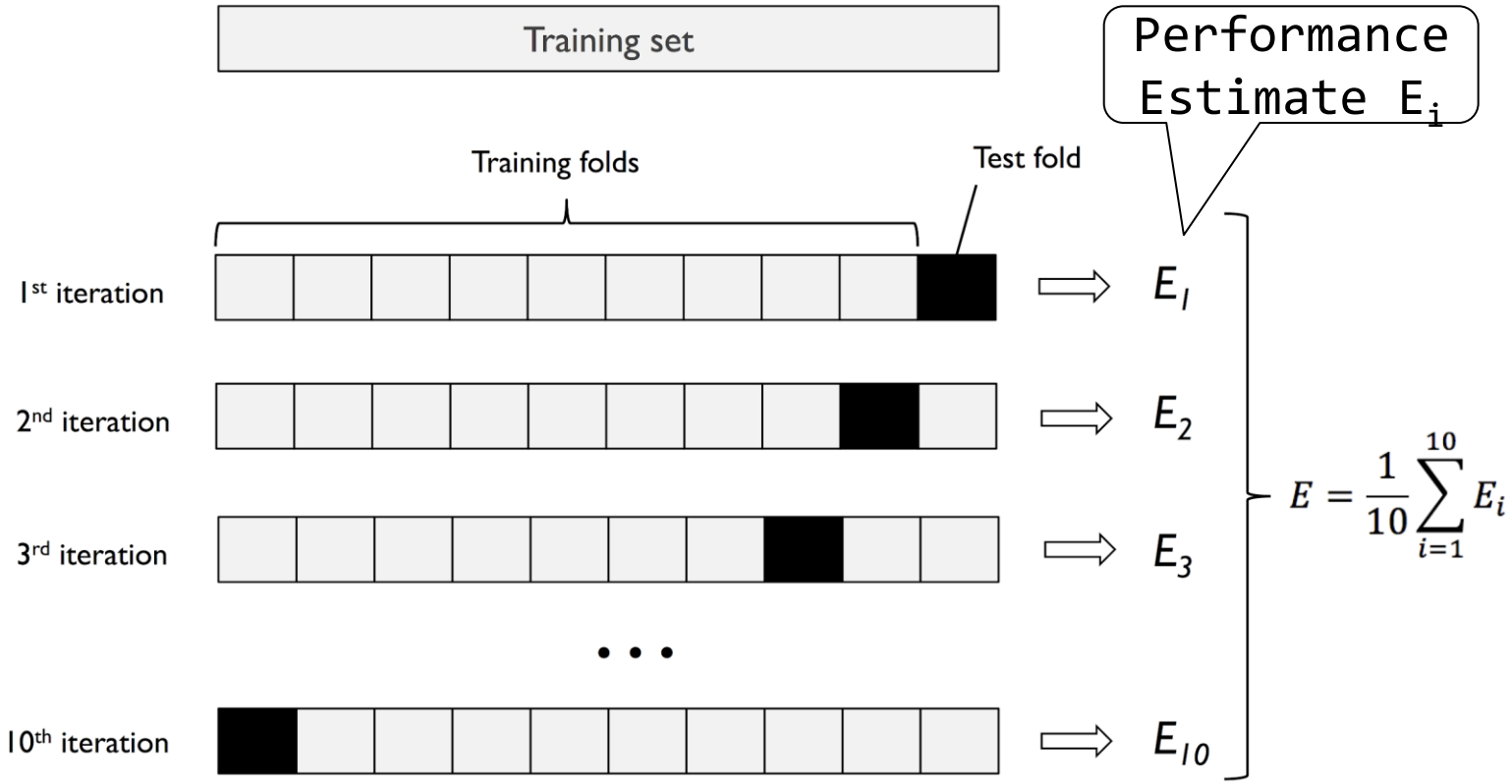
K-Fold Cross-validation

- Create a *K*-fold partition of the dataset
  - For each of *K* experiments, use *K*-1 folds for training and a different fold for testing
    - This procedure is illustrated in the following figure for *K*=4



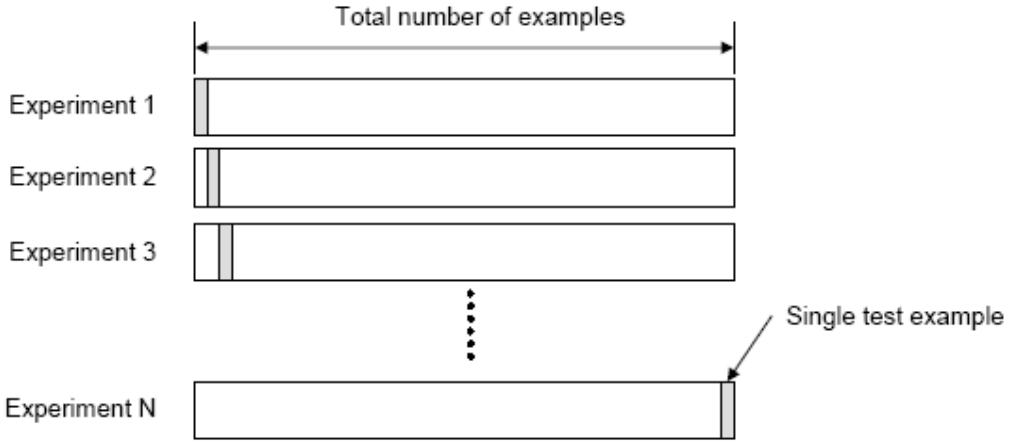
- *K*-Fold Cross validation is similar to Random Subsampling
  - The advantage of *K*-Fold Cross validation is that all the examples in the dataset are eventually used for both training and testing
- The true error is estimated as the average error rate on test examples.

K-Fold Cross-Validation



Leave-one-out Cross Validation

- Leave-one-out is the degenerate case of *K*-Fold Cross Validation, where *K* is chosen as the total number of examples
  - For a dataset with N examples, perform N experiments
  - For each experiment use N-1 examples for training and the remaining example for testing



- As usual, the true error is estimated as the average error rate on test examples.

How many folds are needed?

- With a large number of folds
  - +The bias of the true error rate estimator will be small (the estimator will be very accurate)
  - -The variance of the true error rate estimator will be large
  - -The computational time will be very large as well (many experiments)
- With a small number of folds
  - + The number of experiments and, therefore, computation time are reduced
  - + The variance of the estimator will be small
  - - The bias of the estimator will be large (conservative or larger than the true error rate)
- In practice, the choice of the number of folds depends on the size of the dataset
  - For large datasets, even 3-Fold Cross Validation will be quite accurate
  - For very sparse datasets, we may have to use leave-one-out in order to train on as many examples as possible
- A common choice for K-Fold Cross Validation is K=5 or 10

Outline of Lecture

- Linear regression
  - Estimation criterion
  - Least squares solution
  - Generalization and overfitting
  - Extensions
- Support Vector Machine
  - Maximum Margin Hyperplane
  - The Lagrangian Dual Problem
  - The non-separable case & Non-linear SVMs and kernel methods
  - Optimization techniques
- Cross-Validation
- Performance Evaluation

Precision, Recall, and F-measure (1)

- Suppose the cutoff threshold is chosen to be 0.8. In other words, any instance with posterior probability greater than 0.8 is classified as positive.

- Compute the precision, recall, and F-measure for the model at this threshold value.

Instance	P(+|A)	True Class
1	0.95	+
2	0.93	+
3	0.87	-
4	0.852	-
5	0.851	-
6	0.850	+
7	0.76	-
8	0.53	+
9	0.43	-
10	0.25	+

Precision, Recall, and F-measure (2)

		PREDICTED CLASS	
		Class =Yes	Class= No
ACTUAL CLASS	Class =Yes	(TP) 3	(FN) 2
	Class =No	(FP) 3	(TN) 2

Instance	P(+|A)	True Class
1	0.95	+
2	0.93	+
3	0.87	-
4	0.852	-
5	0.851	-
6	0.850	+
7	0.76	-
8	0.53	+
9	0.43	-
10	0.25	+

Precision, Recall, and F-measure (3)

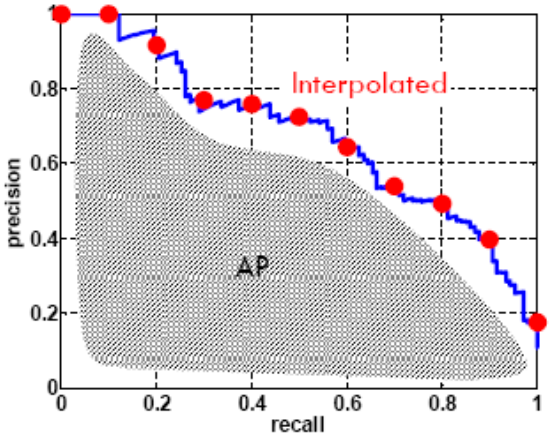
		PREDICTED CLASS	
		Class =Yes	Class= No
ACTUAL CLASS	Class =Yes	(TP) 3	(FN) 2
	Class =No	(FP) 3	(TN) 2

p = TP/(TP+FP)= 3/(3+3) = ½
r = TP/(TP+FN) =3/(3+2) = 3/5
F-measure = 2pr/(p+r)=6/11

TP+FN: the total number of positive samples
FP + TN: the total number of negative samples

Average Precision (1)

- Average Precision (AP):
  - –Averages precision over the entire range of recall
  - – A good score rec recall and high precision
  - – Application-inde



Average Precision (2)

Instance	P(+|A)	True Class
1	0.95	+
2	0.93	+
3	0.87	-
4	0.852	-
5	0.851	-
6	0.85	+
7	0.76	-
8	0.53	+
9	0.43	-
10	0.25	+

How to compute:

$$\frac{1}{5}(1+\frac{2}{2}+\frac{3}{6}+\frac{4}{8}+\frac{5}{10})=0.7$$

How to Construct an ROC curve

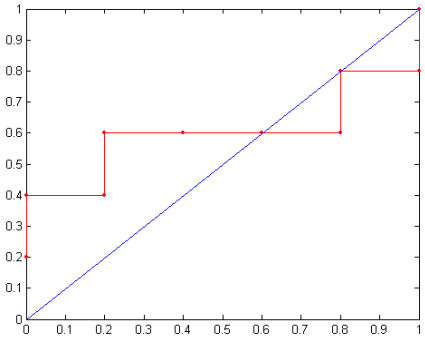
Instance	P(+|A)	True Class
1	0.95	+
2	0.93	+
3	0.87	-
4	0.852	+
5	0.851	-
6	0.85	-
7	0.76	-
8	0.53	+
9	0.43	-
10	0.25	+

- Use classifier that produces posterior probability for each test instance P(+|A)
- Sort the instances according to P(+|A) in decreasing order
- Apply threshold at each unique value of P(+|A)
- Count the number of TP, FP, TN, FN at each threshold
- TP rate, TPR = TP/(TP+FN)
- FP rate, FPR = FP/(FP + TN)

How to construct an ROC curve

	Class	+	-	+	-	-	-	+	-	+	+	
Threshold >=		0.25	0.43	0.53	0.76	0.85	0.851	0.852	0.87	0.93	0.95	1.00
	TP	5	4	4	3	3	3	3	2	2	1	0
	FP	5	5	4	4	3	2	1	1	0	0	0
	TN	0	0	1	1	2	3	4	4	5	5	5
	FN	0	1	1	2	2	2	2	3	3	4	5
→	TPR	1	0.8	0.8	0.6	0.6	0.6	0.6	0.4	0.4	0.2	0
→	FPR	1	1	0.8	0.8	0.6	0.4	0.2	0.2	0	0	0

ROC Curve:



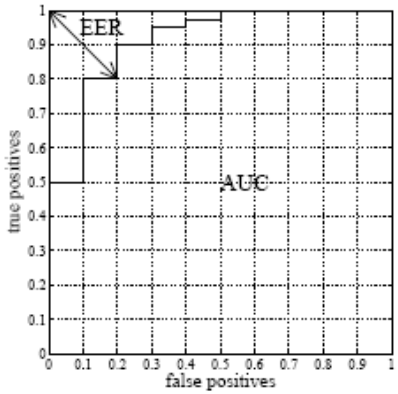
Receiver Operating Characteristic

EER and AUC

- **Equal Error Rate (EER)** measures the **accuracy** at which the number of false positives and false negatives are equal.

Somewhat emphasizes the behaviour of a method at low false positive rates which might be reasonable for a real-world application

- **Area Under Curve (AUC)** measures the total area under the ROC curve.



Penalizes failures across the whole range of false positives, e.g. a method which recognizes some large proportion of instance with zero error but fails on the remaining portion of the data.

Outline of Lecture

- Linear regression
  - Estimation criterion
  - Least squares solution
  - Generalization and overfitting
  - Extensions
- Support Vector Machine
  - Maximum Margin Hyperplane
  - The Lagrangian Dual Problem
  - The non-separable case & Non-linear SVMs and kernel methods
  - Optimization techniques
- Cross-Validation
- Performance Evaluation

Thank you!